

Audit DevOps, déploiement & exploitation

DevOps / Responsable Infrastructure

Projet	CleanUx / brio — marketplace multi-métiers (Laravel 12 + apps mobiles Expo)
Destinataire	DevOps / Responsable Infrastructure
Date d'audit	24 juin 2026
Référence	Réactualisation de l'audit plateforme du 8 juin 2026
Confidentialité	Document interne — confidentiel

Légende sévérité : ● Critique · ● Élevé · ● Moyen · ● Faible · ● Sain / Corrigé

1. Résumé exécutif

Le projet affiche une **culture opérationnelle réelle** (health checks discriminants, heartbeat planifié, garde-fou anti-mock au déploiement, restore-drill bien conçu, runbooks, Sentry & Spatie Backup réellement installés). C'est nettement au-dessus de la moyenne sur l'**intention**.

Mais l'**exécution infrastructure** comporte plusieurs **bloquants de production** : le `Dockerfile` lance `artisan serve` (serveur de développement mono-thread), les sauvegardes ne partent que sur le disque local (pas de copie externe), le driver de file d'attente est incohérent entre l'environnement et les workers, les `migrate --force` s'appliquent automatiquement à chaque déploiement sans sauvegarde préalable, et plusieurs composants sont des points de défaillance unique (Reverb mono-instance, scheduler sans `onOneServer`).

Maturité opérationnelle estimée : 2,5 / 5 — NON prêt pour la production en l'état. Les bloquants sont des correctifs d'infrastructure ciblés (pas une refonte) ; une fois levés, le projet peut viser 4/5.

2. Conteneurisation

2.1 ● Dockerfile = `artisan serve` (serveur de dev mono-thread)

`Dockerfile:1 FROM php:8.3-cli`, `Dockerfile:16-17 CMD ["php","artisan","serve"]`. Pas de PHP-FPM/nginx, image `cli` non destinée au web, mono-thread.

- **Impact** : goulot d'étranglement et indisponibilité sous charge. Ce n'est pas un runtime de production.
- **Remédiation** : Dockerfile multi-stage (build composer/assets → runtime PHP-FPM+nginx ou FrankenPHP/Octane), OPcache activé.

2.2 ● Pas de build d'assets ni d'utilisateur non-root, pas de HEALTHCHECK

`Dockerfile:12 COPY . .` sans `npm run build` ; aucun `USER`, aucun `HEALTHCHECK`. `.dockerignore` minimal (embarque tests, docs, `*.db`).

- **Remédiation** : stage Node pour compiler les assets ; `USER www-data` ; `HEALTHCHECK` sur `/api/health` ; `.dockerignore` exhaustif.

2.3 ● `docker-compose.yml` orienté dev

Secrets en clair (`MYSQL_ROOT_PASSWORD: secret`), bind-mount, ports exposés, Redis sans mot de passe.
Acceptable en dev, à ne pas utiliser en prod.

3. Déploiement & CI/CD

3.1 ● Déploiement « git pull + build sur le serveur » via SSH

`deploy.yml:42-62` : `git pull`, `composer install`, `npm ci`, `npm run build`, `migrate --force` directement sur le serveur de prod. Pas d'artefact immuable, pas de releases atomiques, fenêtre d'incohérence pendant le build, rollback non instantané.

- **Remédiation** : releases symlinkées (dossiers horodatés + `current`), build d'artefact en CI puis transfert, bascule symlink.

3.2 ● Gates d'intégrité argent/RGPD et E2E non bloquants

`ci.yml:88` (`continue-on-error: true` sur `money-integrity-mysql` couvrant Payments/GDPR/KYC) et `ci.yml:150` (E2E). Le déploiement se déclenche sur succès CI (`deploy.yml:14`) — donc **même si ces gates sont rouges**.

- **Impact** : du code financièrement/réglementairement risqué peut atteindre la prod.
- **Remédiation** : rendre ces gates bloquants (issue #5) avant d'autoriser `deploy`.

3.3 ● Drift de version PHP entre test, build et runtime

`composer.json` `^8.2` · CI teste en **8.4** · deploy SSH installe **8.3** · units Supervisor/systemd codent **php8.3** en dur · doc exige **8.5**. « Test ≠ prod ».

- **Remédiation** : figer **une** version PHP de référence partout (CI = runtime), variabiliser le chemin PHP des units.

3.4 ● Workers/scheduler fournis uniquement en `.example`

`deploy/supervisor/*.example`, `deploy/systemd/*.example` : la config des workers de prod n'est pas versionnée comme source de vérité (provisioning manuel, dérive). Pas d'IaC.

Inventaire CI (6 workflows) : `ci.yml` (quality gate + tests + couverture 80 %), `deploy.yml` (prod SSH), `deploy-staging.yml`, `mobile-ci.yml`, `mobile-build.yml` (EAS), `mobile-submit.yml`. ● Points sains : `composer audit` + `npm audit`, PHPStan, Pint, tests mobiles avant build EAS.

4. Files d'attente, scheduler & cache

4.1 ● Driver de queue incohérent (env Redis vs workers `database`)

`config/queue.php:16` défaut `sync` ; `.env.production.example:42` `redis` ; **mais** les workers lancent `queue:work database` en dur. Mismatch silencieux → jobs (webhooks Stripe, KYC, antivirus) **jamais consommés**.

- **Remédiation** : aligner un seul driver (Redis) en env **et** dans les units, via variable.

4.2 🟡 Scheduler : `onOneServer` quasi absent

~35 tâches planifiées, 32 `withoutOverlapping` mais seulement 2 `onOneServer`. Tâches non idempotentes concernées : `payouts:process`, `stripe:reconcile`, `accounting:close-previous-month`, `gdpr:execute-erasures`, `backup:run`.

- **Impact** : au-delà d'un serveur applicatif, chaque serveur exécute ces crons → **double payout, double clôture comptable, double backup, double erasure**.
- **Remédiation** : `onOneServer()` sur toute tâche non idempotente + lock store Redis partagé.

4.3 🟡 Cache/sessions : défauts `file` / `database`, prod attendue en Redis

Si l'env prod n'est pas correctement rempli, fallback non partagé entre instances → incohérence multi-serveur. Redis bien défini par ailleurs. À sécuriser (échec explicite si non configuré).

5. Temps réel (Reverb)

5.1 🟠 Reverb mono-instance = SPOF WebSocket

Scaling désactivé par défaut (`config/reverb.php:41`), un seul process Supervisor (`numprocs=1`), plafond ~1000 connexions sans `ext-uv`. Point de défaillance unique pour chat, présence, suivi de mission. `allowed_origins` ⇒ `['*']` (🟡 à restreindre).

- **Remédiation** : plusieurs instances Reverb derrière LB sticky + scaling Redis + `ext-uv`.

6. Sauvegardes (Spatie Backup)

6.1 🟠 Backups sur le disque `local` uniquement (pas d'offsite)

`config/backup.php:164-166` `'disks' ⇒ ['local']`. Les sauvegardes sont **sur le même serveur** que la base → un sinistre disque/serveur/ransomware emporte données **et** sauvegardes. Pas de règle 3-2-1.

- **Remédiation** : ajouter un disque S3 offsite (déjà disponible dans `config/filesystems.php`), monitorer les deux.

6.2 🟡 Notification backup = placeholder ; restore-drill non planifié

`config/backup.php:225` `'to' ⇒ 'your@example.com'` (échecs silencieux). `backup:restore-drill` existe (excellent, avec garde anti-prod et connexion scratch) mais **n'est pas planifié** → RTO/RPO jamais vérifiés automatiquement.

- **Remédiation** : `to` via env, brancher Slack, forcer `BACKUP_ARCHIVE_PASSWORD`, planifier le drill hebdo en staging.

7. Base de données

7.1 🟠 `migrate --force` automatique sans backup ni mode maintenance

`deploy.yml:55` applique `migrate --force` avant tout health check, sans `backup:run` préalable ni `artisan down`, sur 204 migrations (dont 19 « fix/round » fragiles).

- **Remédiation** : `backup:run` / snapshot **immédiatement avant** `migrate`, `artisan down` pour migrations à risque, `migrate --pretend` validé en CI.

8. Secrets, scalabilité & rollback

8.1 ● Secrets uniquement via `.env` plat (pas de vault)

Secrets de prod (Stripe live, KYB, S3, DB) en fichier plat sur le serveur, sans rotation centralisée ni audit d'accès ; le guide backup inclut `.env` dans les archives.

- **Remédiation** : gestionnaire de secrets (AWS Secrets Manager/Vault) injecté au runtime, exclure `.env` des backups non chiffrés.

8.2 ● Multiples SPOF & statelessness conditionnelle

Reverb mono-process, app mono-thread, crons sans `onOneServer`, cache/session en fallback local, backups local-only → architecture non prête à scaler horizontalement de façon sûre. Rollback partiel (pas de releases atomiques).

9. Mobile (EAS / stores)

● CI mobile (lint/typecheck/tests), build EAS gated par `npm test`, profils par env, submit séparé. ● **F11.1** : identifiants de soumission store en placeholders (`appleId`, `ascAppId`, `appleTeamId`) → `mobile-submit.yml` échouera tel quel. ● Pas d'OTA (`eas update`).

10. Points sains

- ● `/api/health` discriminant (dépendances dures → 503 vs soft), liveness/readiness, heartbeat planifié.
- ● `ops:check-providers --strict` bloque le déploiement si un provider mock est actif en prod (excellent garde-fou).
- ● Sentry, Spatie Backup, Reverb, Cashier, Sanctum réellement installés ; restore-drill robuste ; chiffrement backup AES-256 activable ; secrets `.env` gitignores ; `withoutOverlapping` quasi systématique.

11. Plan d'action priorisé & verdict

#	Action	Priorité
1	Dockerfile prod (FPM/Octane multi-stage, non-root, HEALTHCHECK, build assets)	●
2	Backups offsite chiffrés (S3) + monitoring + notif réelle	●
3	Aligner le driver de queue (Redis) env + units	●
4	<code>backup:run</code> avant <code>migrate</code> + <code>artisan down</code> ; gates argent/RGPD bloquants	●
5	<code>onOneServer()</code> sur crons non idempotents + lock Redis	●
6	Reverb multi-instance + scaling Redis ; aligner versions PHP	●
7	Secrets externalisés (vault) ; identifiants store EAS réels	●

Verdict : maturité 2,5/5. Bonne intention opérationnelle, mais 8 bloquants Critiques/Élevés à lever avant go-live. Tous sont des correctifs d'infrastructure ciblés.

Réserve : audit statique du dépôt ; aucun déploiement réel observé, RTO/RPO non mesurés en conditions réelles.